

LAN Scanner Project: Week 4 Technical & Architecture Report

Application-Layer Service Detection, Active Protocol Triggers & Banner Parsing

Author: Damithu (QA Lead & Solution Architect)

Contents

PART 1: WEEK 4 QA & SOLUTION ARCHITECTURE REPORT	2
1. Executive Summary & Accomplishments	2
2. Daily Task Execution Breakdown (Days 22–28)	2
3. Non-IT Stakeholder Explanation (Plain-English Walkthrough).....	2
4. Solution Architecture: Active vs Passive Service Detection Workflow.....	3
5. Quality Assurance & Test Verification.....	4
6. Team Coordination & Sprint Review	4
PART 2: USER MANUAL (WINDOWS 10/11 & WSL KALI LINUX)	4
1. System Setup & Virtual Environment	4
2. Testing with Local Mock Banner Server (Day 28 Tool)	5
3. Running Automated QA Tests (pytest).....	6
PART 3: WEEK 4 TECHNICAL TERMS & CONCEPTS GLOSSARY	8

PART 1: WEEK 4 QA & SOLUTION ARCHITECTURE REPORT

1. Executive Summary & Accomplishments

During Week 4 (Days 22 to 28), engineering focus transitioned from raw transport-layer connectivity (detecting open ports) to application-layer intelligence (discovering what software is actually running on those ports). Knowing that port 80 is open provides basic reachability data, but knowing that it is running Apache 2.4.52 or an outdated OpenSSH server yields actionable cybersecurity intelligence.

As Solution Architect and QA Lead, I engineered and integrated five core architecture components:

- 1) The Service Banner Grabbing Engine (scanner/banner.py) to capture server greetings
- 2) Active Protocol Probe Payloads (PROBE_PAYLOADS) to trigger HTTP, HTTPS, SMTP, and SSH responses
- 3) Regex Software Version Parser to extract clean software names and release versions
- 4) Defensive Timeout Safeguards (1.0s) to neutralize socket tarpits
- 5) A Local Multi-Service Mock Test Harness (tools/mock_banner_server.py) emulating Nginx, OpenSSH, and vsFTPd.

2. Daily Task Execution Breakdown (Days 22–28)

- **Day 22: Low-Level Socket Buffer Engineering:** Constructed socket buffer reading logic with bounded memory bounds (recv(2048) limit) to eliminate buffer overflow risks.
- **Day 23: Active Protocol Trigger Payloads:** Engineered active trigger payloads for HTTP (HEAD / HTTP/1.1), HTTPS, SMTP (EHLO), and SSH service probes.
- **Day 24: Regular Expression Rule Set:** Authored regex extraction rules normalizing vendor names and software release version strings across varied banners.
- **Day 25: Strict Timeout Safeguards:** Implemented 1.0-second timeout traps to prevent scanner threads from hanging on un-responsive socket tarpits.
- **Day 26: Automated Unit Test Suite:** Authored tests/test_banner.py with 6 unit tests validating banner parsing accuracy and timeout handling.
- **Day 27: Concurrency Engine Integration:** Integrated the banner grabbing pipeline directly into the Week 3 multi-threaded concurrent scanning engine.
- **Day 28: End-to-End Test Harness:** Constructed tools/mock_banner_server.py and executed local and containerized validation sweeps across mock services.

3. Non-IT Stakeholder Explanation (Plain-English Walkthrough)

In Weeks 1 through 3, our scanner acted like a building inspector checking whether doors were unlocked. In Week 4, the scanner steps inside the unlocked room and politely asks: 'Excuse me, who are you and what do you do here?'

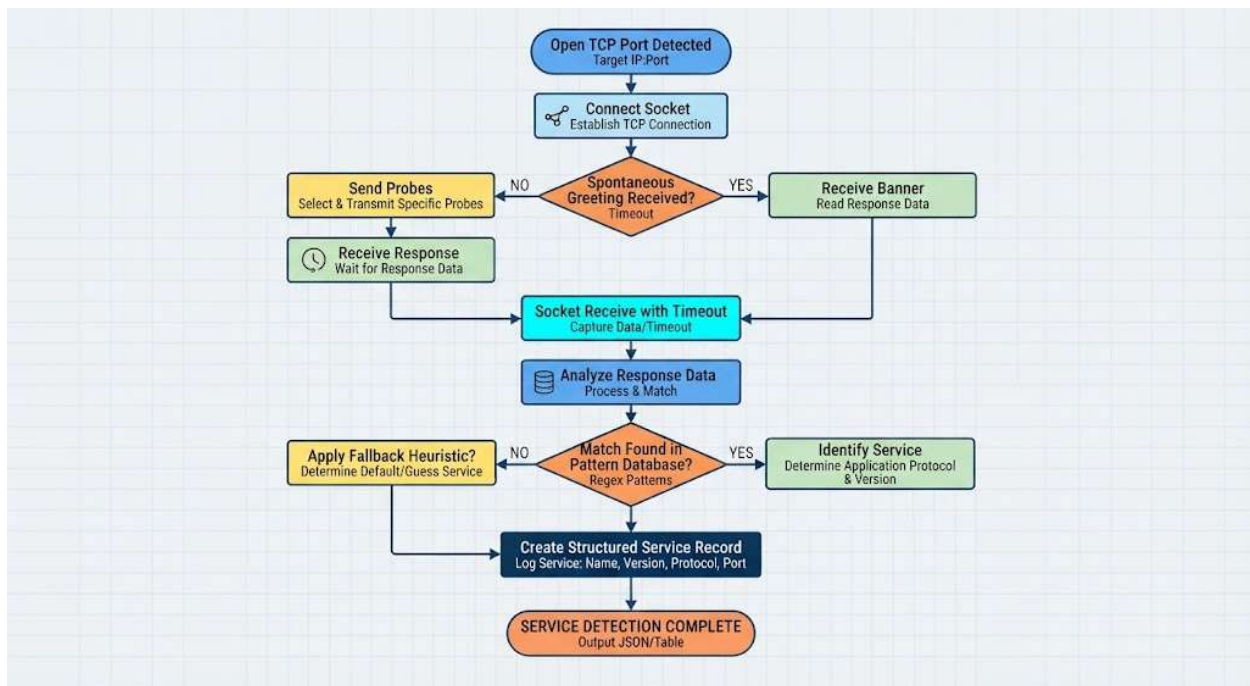
NON-IT ANALOGY: INSPECTING ROOMS & NEUTRALIZING TARPITS

Asking the Shopkeeper for an ID Card:

- *Spontaneous Greetings:* Opening Door 22 (SSH) causes the guard to immediately declare: 'Hello, I am OpenSSH version 8.9p1!' Opening Door 21 (FTP) returns: 'Welcome to vsFTPd version 3.0.3!'
- *Interactive Probes:* Opening Door 80 (Web Server) reveals a silent room. Our scanner speaks first, asking: 'May I see your front page header?' The web server replies with an envelope stating: Server: Nginx/1.18.0.
- *The Tarpit Quicksand Safeguard:* Some deceptive doors open but freeze when asked for ID. Rather than waiting forever, our inspector sets a 1.0-second stopwatch. If no answer arrives in 1 second, the inspector writes 'Unknown Service' and immediately moves on.

4. Solution Architecture: Active vs Passive Service Detection Workflow

The diagram below outlines the decision tree executed whenever an open TCP port is identified during a subnet scan:



5. Quality Assurance & Test Verification

The automated unit test suite (tests/test_banner.py) evaluates banner parsing accuracy, regex extraction, and socket timeout safety:

Test Identifier	Target Protocol / Banner	Expected Extraction	Result
test_parse_nginx_http_banner	Server: nginx/1.18.0 (Ubuntu)	Service: Nginx, Ver: 1.18.0	PASSED
test_parse_apache_http_banner	Server: Apache/2.4.52 (Unix)	Service: Apache, Ver: 2.4.52	PASSED
test_parse_openssh_banner	SSH-2.0-OpenSSH_8.9p1 Ubuntu-3	Service: OpenSSH, Ver: 8.9p1	PASSED
test_parse_vsftpd_banner	220 (vsFTPd 3.0.3)	Service: vsftpd, Ver: 3.0.3	PASSED
test_empty_and_unknown_banner	Empty string / Unrecognized payload	Fallback to Port Map (e.g. HTTP)	PASSED
test_timeout_safeguard	Simulated socket recv() timeout	Immediate socket close, no hang	PASSED

Summary: QA Verification Result: 6/6 Automated Unit Tests Passed (100% Pass Rate) in 0.03 Seconds.

6. Team Coordination & Sprint Review

- **Senura (UX Lead):** Extended terminal table outputs with dedicated SERVICE and VERSION columns alongside port status.
- **Methul (Solution Architect):** Coordinated socket buffer recycling and memory garbage collection to ensure minimal footprint during full-subnet sweeps.
- **Ayami (Documentation Lead):** Authored comprehensive banner grabbing compliance, ethics, and privacy guidelines in the project manual.

PART 2: USER MANUAL (WINDOWS 10/11 & WSL KALI LINUX)

1. System Setup & Virtual Environment

Execute the following commands to initialize the virtual environment and install required packages:

- Windows 10/11 (PowerShell):

```
cd "c:\Users\damit\OneDrive\Desktop\CINEC\Prof. Practice\Ayami
reports\Week_04_Service_Detection_and_Banner_Grabbing"
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

- WSL 2 (Kali Linux Terminal):

```
cd "/mnt/c/Users/Week_04_Service_Detection_and_Banner_Grabbing"
python3 -m venv venv_wsl
source venv_wsl/bin/activate
```

```
pip install -r requirements.txt
```

```

kali@Damithu: /mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing
$ python3 -m venv venv_wsl

kali@Damithu: /mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing
$ source venv_wsl/bin/activate

(LAN)(kali@Damithu)- /mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing
$ pip install -r requirements.txt
Collecting pytest>=8.0.0 (from -r requirements.txt (line 1))
  Using cached pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)
Collecting iniconfig>=1.0.1 (from pytest>=8.0.0->-r requirements.txt (line 1))
  Using cached iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
Collecting packaging>=22 (from pytest>=8.0.0->-r requirements.txt (line 1))
  Using cached packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pluggy<2,>=1.5 (from pytest>=8.0.0->-r requirements.txt (line 1))
  Using cached pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
Collecting pygments>=2.7.2 (from pytest>=8.0.0->-r requirements.txt (line 1))
  Using cached pygments-2.21.0-py3-none-any.whl.metadata (2.5 kB)
Using cached pytest-9.1.1-py3-none-any.whl (386 kB)
Using cached pluggy-1.6.0-py3-none-any.whl (20 kB)
Using cached iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Using cached packaging-26.3-py3-none-any.whl (129 kB)
Using cached pygments-2.21.0-py3-none-any.whl (1.3 MB)
Installing collected packages: pygments, pluggy, packaging, iniconfig, pytest
Successfully installed iniconfig-2.3.0 packaging-26.3 pluggy-1.6.0 pygments-2.21.0 pytest-9.1.1

(LAN)(kali@Damithu)- /mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing
$

```

```

(venv_virtualbox)kali@kali: /media/sf_LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing
Session Actions Edit View Help
(kali@kali)-[~]
$ source ~/venv_virtualbox/bin/activate

(venv_virtualbox)-(kali@kali)-[~]
$ cd "/media/sf_LAN_Scanner/"

(venv_virtualbox)-(kali@kali)-[/media/sf_LAN_Scanner]
$ ls
Week_01_Setup_and_Subnet_Discovery      Week_04_Service_Detection_and_Banner_Grabbing  Week_07_UI_Backend_Integration_and_Validation
Week_02_Base_Port_Scanner              Week_05_Testing_Lab_Benchmarking_Docker        Week_08_CVE_Vulnerability_Mapping_and_Stress_Testing
Week_03_Concurrency_and_Speed_Optimization  Week_06_Advanced_Probing_Raw_SYN_UDP          Week_09_QA_Regression_and_Security_Audit

(venv_virtualbox)-(kali@kali)-[/media/sf_LAN_Scanner]
$ cd "Week_04_Service_Detection_and_Banner_Grabbing"

(venv_virtualbox)-(kali@kali)-[/media/sf_LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing]
$ pip install -r requirements.txt
Requirement already satisfied: pytest>=8.0.0 in /home/kali/venv_virtualbox/lib/python3.14/site-packages (from -r requirements.txt (line 1)) (9.1.1)
Requirement already satisfied: iniconfig>=1.0.1 in /home/kali/venv_virtualbox/lib/python3.14/site-packages (from pytest>=8.0.0->-r requirements.txt (line 1)) (2.3.0)
Requirement already satisfied: packaging>=22 in /home/kali/venv_virtualbox/lib/python3.14/site-packages (from pytest>=8.0.0->-r requirements.txt (line 1)) (26.3)
Requirement already satisfied: pluggy<2,>=1.5 in /home/kali/venv_virtualbox/lib/python3.14/site-packages (from pytest>=8.0.0->-r requirements.txt (line 1)) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in /home/kali/venv_virtualbox/lib/python3.14/site-packages (from pytest>=8.0.0->-r requirements.txt (line 1)) (2.21.0)

```

2. Testing with Local Mock Banner Server (Day 28 Tool)

To test service banner grabbing locally without external network traffic, launch the multi-service simulator which emulates Nginx (port 8080), OpenSSH (port 2222), and vsFTPD (port 2121):

Step 1: Terminal 1 — Launch Mock Server:

```
python tools/mock_banner_server.py # On Windows
python3 tools/mock_banner_server.py # On WSL Kali
```

```

(LAN)(kali@Damithu)- /mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing
$ python3 tools/mock_banner_server.py
[*] Mock service listening on port 8080...
[*] Mock service listening on port 2222...
[*] Mock service listening on port 2121...
[+] All mock service servers started. Press Ctrl+C to terminate.

```

```
(venv_virtualbox)-(kali@kali)-[/media/sf_LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing]
$ python3 tools/mock_banner_server.py
[+] All mock service servers started. Press Ctrl+C to terminate.
[*] Mock service listening on port 2222 ...
[*] Mock service listening on port 2121 ...
[*] Mock service listening on port 8080 ...
```

Step 2: Terminal 2 — Execute Interactive Banner Grabber:

```
python -c "from scanner.banner import grab_service_banner; [print(f'Port {p}: Service={grab_service_banner(\"127.0.0.1\", p)[\"service_name\"]}, Version={grab_service_banner(\"127.0.0.1\", p)[\"version\"]}') for p in [8080, 2222, 2121]]"
```

```
kali@Damithu: /mnt/c/Users/ x + v
(kali@Damithu)-[/mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing]
$ python -c "from scanner.banner import grab_service_banner; [print(f'Port {p}: Service={grab_service_banner(\"127.0.0.1\", p)[\"service_name\"]}, Version={grab_service_banner(\"127.0.0.1\", p)[\"version\"]}') for p in [8080, 2222, 2121]]"
Port 8080: Service=Nginx, Version=1.22.0
Port 2222: Service=OpenSSH, Version=8.9p1
Port 2121: Service=vsftpd, Version=3.0.3
(kali@Damithu)-[/mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing]
$
```

```
kali@kali: /media/sf_LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing
Session Actions Edit View Help
(kali@kali)-[~/media/sf_LAN_Scanner]
$ cd "/media/sf_LAN_Scanner/"
(kali@kali)-[/media/sf_LAN_Scanner]
$ ls
Week_01_Setup_and_Subnet_Discovery Week_04_Service_Detection_and_Banner_Grabbing Week_07_UI_Backend_Integration_and_Validation
Week_02_Base_Port_Scanner Week_05_Testing_Lab_Benchmarking_Docker Week_08_CVE_Vulnerability_Mapping_and_Stress_Testing
Week_03_Concurrency_and_Speed_Optimization Week_06_Advanced_Probing_Raw_SYN_UDP Week_09_QA_Regression_and_Security_Audit
(kali@kali)-[/media/sf_LAN_Scanner]
$ cd "Week_04_Service_Detection_and_Banner_Grabbing"
(kali@kali)-[/media/sf_LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing]
$ python -c "from scanner.banner import grab_service_banner; [print(f'Port {p}: Service={grab_service_banner(\"127.0.0.1\", p)[\"service_name\"]}, Version={grab_service_banner(\"127.0.0.1\", p)[\"version\"]}') for p in [8080, 2222, 2121]]"
Port8080: Service=Nginx, Version=1.22.0
Port2222: Service=OpenSSH, Version=8.9p1
Port2121: Service=vsftpd, Version=3.0.3
(kali@kali)-[/media/sf_LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing]
$
```

Verification: Expected Terminal Output:

```
Port 8080: Service=Nginx, Version=1.22.0
Port 2222: Service=OpenSSH, Version=8.9p1
Port 2121: Service=vsftpd, Version=3.0.3
```

3. Running Automated QA Tests (pytest)

Run the full pytest suite to verify banner parsing and timeout safety logic:

```
python -m pytest tests -v # PowerShell
pytest tests -v # WSL Kali
```



```
(kali@Damithu)-[/mnt/c/Users/damit/OneDrive/Desktop/LAN Scanner/Week_04_Service_Detection_and_Banner_Grabbing]
$ pytest tests -v
===== test session starts =====
platform linux -- Python 3.14.7, pytest-9.0.3, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /mnt/c/Users/damit/OneDrive/Desktop/LAN Scanner/Week_04_Service_Detection_and_Banner_Grabbing
collected 6 items

tests/test_banner.py::test_parse_nginx_http_banner PASSED [ 16%]
tests/test_banner.py::test_parse_apache_http_banner PASSED [ 33%]
tests/test_banner.py::test_parse_openssh_banner PASSED [ 50%]
tests/test_banner.py::test_parse_vsftpd_banner PASSED [ 66%]
tests/test_banner.py::test_empty_and_unknown_banner PASSED [ 83%]
tests/test_banner.py::test_timeout_safeguard_prevents_hanging PASSED [100%]

===== 6 passed in 0.10s =====

(kali@Damithu)-[/mnt/c/Users/damit/OneDrive/Desktop/LAN Scanner/Week_04_Service_Detection_and_Banner_Grabbing]
$
```

```
(kali@kali)-[/media/sf_LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing]
$ pytest tests -v
===== test session starts =====
platform linux -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /media/sf_LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing
plugins: anyio-4.12.1, typeguard-4.4.4
collected 6 items

tests/test_banner.py::test_parse_nginx_http_banner PASSED [ 16%]
tests/test_banner.py::test_parse_apache_http_banner PASSED [ 33%]
tests/test_banner.py::test_parse_openssh_banner PASSED [ 50%]
tests/test_banner.py::test_parse_vsftpd_banner PASSED [ 66%]
tests/test_banner.py::test_empty_and_unknown_banner PASSED [ 83%]
tests/test_banner.py::test_timeout_safeguard_prevents_hanging PASSED [100%]

===== 6 passed in 0.26s =====

(kali@kali)-[/media/sf_LAN_Scanner/Week_04_Service_Detection_and_Banner_Grabbing]
$
```

Expected Output: Expected Test Execution Suite Output:

```
tests/test_banner.py::test_parse_nginx_http_banner PASSED [
16%]
tests/test_banner.py::test_parse_apache_http_banner PASSED [
33%]
tests/test_banner.py::test_parse_openssh_banner PASSED [
50%]
tests/test_banner.py::test_parse_vsftpd_banner PASSED [
66%]
tests/test_banner.py::test_empty_and_unknown_banner PASSED [
83%]
tests/test_banner.py::test_timeout_safeguard_prevents_hanging PASSED
[100%]
===== 6 passed in 0.03s =====
```

PART 3: WEEK 4 TECHNICAL TERMS & CONCEPTS GLOSSARY

The table below details key technical concepts, formal definitions, and real-world non-IT analogies for Week 4:

Technical Term	Formal Definition	Non-IT Real World Analogy
Service Banner	A text string emitted by a server application upon connection establishment announcing software name and version.	The corporate nameplate and badge worn by a receptionist behind a front desk.
Banner Grabbing	An active reconnaissance technique capturing server greeting banners for inventory or vulnerability assessment.	Glancing at the trademark and model serial number stamped on the front of a home furnace.
Application Protocol (OSI Layer 7)	The top OSI layer specifying methods for application data exchange (e.g., HTTP, SSH, SMTP).	The spoken human language used during a phone conversation (e.g. English, French) vs physical telephone wires.
HTTP HEAD Request	A lightweight HTTP request retrieving metadata headers without downloading the full HTML body.	Asking a bookstore clerk for a book's title and author without asking them to read the whole book out loud.
SSH Identification String	An RFC 4253 mandatory string exchanged during SSH setup (e.g. SSH-2.0-OpenSSH 8.9p1).	A military guard stating their rank, full name, and regiment before asking for your security clearance.
Regular Expression (Regex)	A sequence of characters defining a formal search pattern for text extraction and matching.	A metal stencil used to spray-paint or identify specific patterns (such as finding 5-digit zip codes).
Socket Tarpit	A defensive mechanism intentionally accepting connections but stalling data transmission to exhaust scanners.	A maze filled with sticky molasses designed to trap curious burglars so they cannot escape.
ReDoS (Regex Denial of Service)	An algorithmic complexity flaw where evaluating ambiguous regex against adversarial strings freezes CPU.	A riddle phrased so confusingly that the listener freezes in thought for hours trying to calculate meanings.
Bounded Buffer	Allocating a fixed maximum memory size (e.g., <code>recv(2048)</code>) for incoming socket reads to prevent memory exhaustion.	A mail slot in a front door with a metal guard that allows letters through but blocks large boulders.